

Kreaturen-Archiv

Eine Desktop-App mit JSON-Daten, Suche, Filter und Animation

Python-Projekt · Tag 4

Worum geht es?

Du baust eine **Desktop-App**, die eine größere Datenmenge aus einer JSON-Datei einliest und interaktiv darstellt.

- `kreaturen.json`: Titel, Untertitel und **30 erfundene Kreaturen**
- Jede Kreatur hat einen **Typ**, **drei Werte** und eine **Beschreibung**
- Die Oberfläche baust **du selbst**
- Vorbereitet ist nur das Fundament: Laden, zwei Hilfsfunktionen, Hauptprogramm

Das kann die App am Ende

Funktion	Was passiert
Suchen	Namen eintippen, auf „Suchen“ klicken
Filtern	Typ-Buttons: Alle, Feuer, Wasser, Wald, Schatten, Eis
Karten erzeugen	Pro Treffer ein Button, der Zähler zeigt die Anzahl
Details	Klick auf Karte: Beschreibung, Werte, animierte Balken
Zufall	Zufällige Kreatur auswählen
Stärkste	Stärkste Kreatur der aktuellen Auswahl
Duell	Kampf gegen einen zufälligen Gegner

Die Dateien

Datei	Inhalt
creaturen.json	Die Daten
projekt.py	Dein Startpunkt: Fundament + nummerierte Aufgaben
projekt_loesung.py	Die fertige Lösung

Alle Dateien liegen im **selben Ordner**.

Wie läuft das Programm ab?

1. `daten_laden` liest die JSON und berechnet den **Gesamtwert** jeder Kreatur
2. `main` erstellt das Fenster, `oberflaeche_bauen` setzt alle Widgets hinein
3. `ergebnisse_aktualisieren` **filtert**, löscht alte Karten, erzeugt neue
4. Klick auf eine Karte → Button ruft `detail_zeigen` auf
5. `animation_schritt` zeichnet neu und plant sich mit `window.after(20, ...)` **selbst wieder ein**, bis `fortschritt` 100 erreicht

Idee 1

- Alle Kreaturen: `daten["kreaturen"]`
- Die Liste `sichtbare` enthält nur, was gerade zu **Suche und Filter** passt
- Bei jeder Änderung wird `sichtbare` **neu aufgebaut**

`daten["kreaturen"]` (alle 30) —Filter + Suche—> `sichtbare` (z. B. 7) —Schleife—> Karten (7 Buttons)

Idee 2

Für **jede Kreatur** in `sichtbare` entsteht per Schleife ein Button (Abschnitt 32).

Vor dem Neuaufbau müssen die alten Karten **weg**:

```
for k in karten:  
    k.destroy()  
karten = []           # Liste leeren!
```

Merksatz: lambda bei Buttons in Schleifen

Ohne Trick würde **jeder** Button denselben Wert benutzen.

```
command=lambda t=typ: typ_waehlen(t)
```

Das `t=typ` „friert“ den Wert **im Moment des Erzeugens** ein, sodass jeder Button eigene Werte hat.

Idee 3

- `fortschritt` läuft von **0 bis 100**
- In jedem Schritt: Wert $\times$ `fortschritt / 100`
- Bei `fortschritt = 50` ist jeder Balken **halb so lang**
- Der Balken ist nur **Text**: `"■" * 16`
- Die Gesamtzahl im Canvas ändert sich mit `itemconfig` wie beim Pomodoro-Timer

`"■" * 8` `#` →  (halber Balken bei 16 Stellen)

Los geht's!